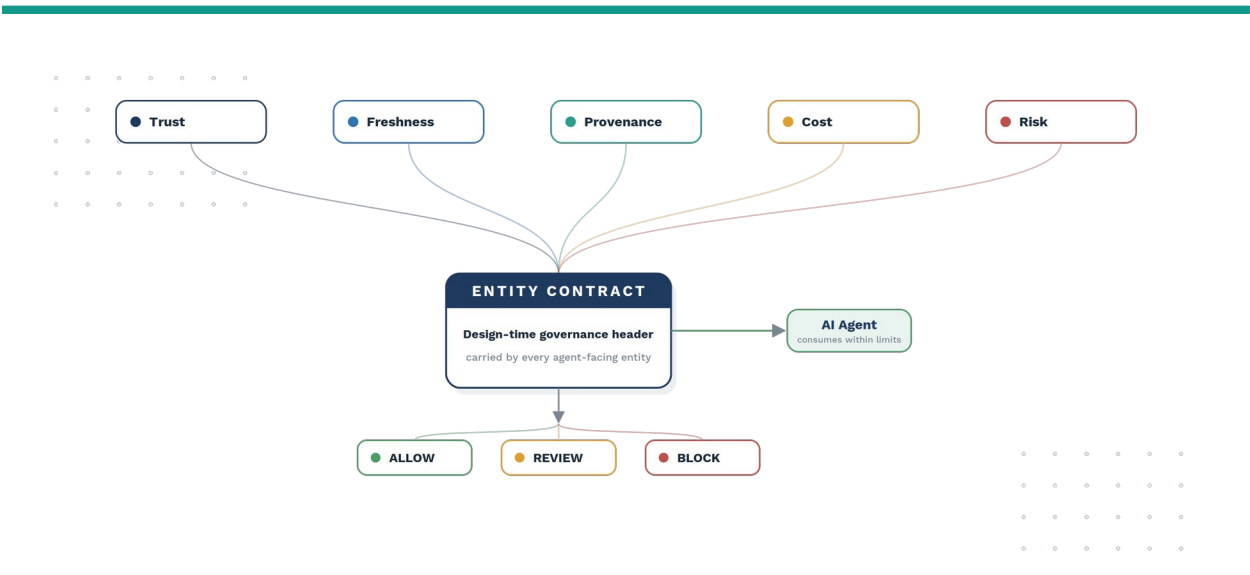


Agent Contract

Data Modeling

A design-time methodology for building agent-safe enterprise data products



Ravi Kiran Pagidi

Senior AI Data Engineer · Enterprise Data & Agentic AI Systems

ACDM v1.0 · Public Draft for Expert Review · June 2026

Prepared for expert review, practitioner feedback, and independent publication.

Contents

1. Executive Summary.....	3
2. Abstract.....	3
3. Introduction.....	4
4. Motivating Scenario.....	4
5. Background and Related Work.....	5
6. How ACDM Differs from Traditional Data Contracts.....	7
7. The ACDM Methodology.....	8
8. Core Primitive: The Entity Contract.....	9
9. The Five Agent Properties.....	10
10. ACDM Composition Rules.....	12
11. Agent-Consumption Permission Model.....	13
12. ACDM Maturity Model.....	15
13. Reference Architecture.....	16
14. Reference Implementation.....	16
15. Worked Example: Synthetic Validation Study.....	17
16. Practical Adoption Guide.....	19
17. Relationship to Prior Art and Limitations.....	20
18. Future Work.....	20
19. Conclusion.....	21
References.....	21
Appendix A. Publication-Readiness Checklist.....	22
Appendix B. Entity Contract Full Template.....	22
Appendix C. Validator and Maturity Worksheet.....	23

1. Executive Summary

Enterprises are moving quickly from asking whether AI agents can query their data to asking whether agents can act on it. That second question is harder, and it exposes a gap in how data is modeled. Traditional data modeling assumed a human analyst or a deterministic application stood between the data and any consequential action. Agentic AI compresses that gap. An autonomous or semi-autonomous agent reads enterprise tables, reasons over them in natural language, combines sources, forms recommendations, and increasingly triggers workflows, often with limited human review.

Schemas, catalogs, and lineage were built for that earlier consumer, and on their own they are not enough for an agent. A schema describes the shape of a value, not whether it can be trusted. Lineage as practiced today is computed after data moves, which helps an investigator but not an agent deciding in the moment. Catalog documentation is written for people who exercise judgment. None of these tells an agent, at the instant it consumes an entity, whether the data is trustworthy, fresh, traceable, affordable to use, and safe for a given class of action.

Agent Contract Data Modeling proposes a design-time answer. It requires each agent-facing entity to carry an **Entity Contract** that declares five agent-relevant properties: trust, freshness, provenance, cost, and risk. It defines conservative composition rules for how those properties combine when entities are joined or enriched, an agent-consumption permission model that maps the resolved properties to a concrete action ceiling, a five-level maturity model for staged adoption, and a validator that certifies the result and emits an audit record. The methodology does not invent provenance, lineage, data contracts, or governance. Its contribution is the practical unification of these ideas into one design-time discipline for agent-safe data products.

This white paper is written for senior data engineers, data and platform architects, AI governance teams, and applied researchers. The practical value is concrete: model one agent-facing data product, classify its entities, apply the rules, and you can see how many unsafe agent actions the contracts prevent *before* they reach production, with an audit trail a governance team can defend.

2. Abstract

Traditional data modeling methods were built for databases, reporting, and human analysts, and they assume a person sits between the data and any consequential action. Agentic AI reduces or removes that checkpoint. Autonomous and semi-autonomous agents read enterprise data, reason over it, recommend actions, and in some cases trigger workflows without review. These agents need more than a schema, a lineage graph, and a catalog description. They need to know, at the moment they consume an entity, whether the data is trustworthy, fresh, traceable, affordable to use, and safe for a given class of action. Today that information is usually inferred at runtime, often incorrectly, and rarely recorded in a way that explains why an agent acted as it did. Agent Contract Data Modeling (ACDM) is a design-time methodology that addresses this gap. It requires each agent-facing entity to carry an Entity Contract declaring five properties: trust tier, freshness policy, provenance binding, cost class, and risk class. It defines composition rules for how those properties combine across joins and aggregations, and a

permission model that maps the result to a concrete action ceiling, from blocked through autonomous action. A five-level maturity model and a validator-based certification approach make the methodology measurable and auditable. ACDM does not claim to invent provenance, lineage, metadata tagging, trust propagation, or data contracts. It builds on that prior work and contributes their unification into a single design-time methodology for agent-safe enterprise data products.

Naming note: the acronym ACDM also appears in other fields, such as clinical data management. To avoid ambiguity, this paper uses the full name, Agent Contract Data Modeling, and uses the short form only after the full name has been introduced.

3. Introduction

For three decades, the consumer of a well-modeled data set was a person or a deterministic program. Kimball gave us facts and dimensions so analysts could slice a warehouse. Inmon gave us the enterprise data warehouse. Data Vault gave us auditable, history-preserving structures. More recently, data mesh and the data-product movement pushed ownership and quality toward domain teams. Every one of these advances optimized for the same kind of consumer: one that asks questions and reads answers, with a human deciding what to do next.

Agentic AI introduces a consumer that behaves differently. An agent does not just read a row. It interprets the row in natural language, combines it with other sources, forms a recommendation, and increasingly takes an action such as opening a case, sending a message, adjusting a limit, or calling another system. The agent compresses the gap between data and consequence that a human analyst used to occupy. When that gap closes, the assumptions baked into traditional modeling stop holding.

Schema alone is not enough, for three reasons. First, a schema describes shape, not standing. It tells an agent that a column is a decimal; it does not tell the agent whether that decimal came from a system of record or a scraped estimate. Second, lineage as practiced today is retrospective: most lineage is computed after data moves, which helps an investigator but does nothing for an agent deciding in the moment. Third, documentation is written for humans who exercise judgment. An agent does not read a wiki page and decide to be careful. It needs machine-readable limits.

ACDM responds to this shift by treating the agent as a first-class consumer with its own modeling requirements. The premise is simple to state and demanding to practice: **if an agent is allowed to act on an entity, the conditions for that action must be declared when the entity is modeled, not discovered after something goes wrong.**

4. Motivating Scenario

Consider a generic regulated financial-services setting. No real institution or system is described here; the entities are illustrative. An AI assistant helps service representatives by recommending the next best action for a member who calls in. To do that, the agent reads four entities: `customer_profile` (identity and relationship attributes from the system of record), `transaction_summary` (rolling aggregates materialized nightly), `third_party_risk_score` (a fraud and credit signal purchased from an external vendor),

and `product_eligibility` (a derived table joining the others to flag which products a member qualifies for).

On a good day, the agent reads `product_eligibility`, recommends an action, and the representative accepts it. The failure modes appear when the underlying properties go unmodeled.

Stale data. The `third_party_risk_score` was last refreshed eleven days ago because the vendor feed silently failed. Nothing in the schema tells the agent the score is stale, so it treats an eleven-day-old fraud signal as current and recommends raising a limit.

Trust degradation. The `product_eligibility` table joins a high-trust profile with a best-effort estimate. Because the join produced a clean table, the agent assumes the whole row is high-trust. The weakest input quietly set the real reliability of the output, but nothing recorded that.

Missing provenance. When a complaint is filed, no one can say which source drove the recommendation, because the lineage was never required to resolve to a named system of record.

Unbounded cost and high-risk action. The agent re-materializes an expensive aggregate on every call, and it takes an action on a high-sensitivity attribute that should always have required human review. Neither limit was expressed anywhere the agent could read.

Every one of these failures is a modeling omission, not a model defect. The agent behaved reasonably given what it was told. **It was told too little.** ACDM exists to make sure the entity carries what the agent needs to behave safely.

5. Background and Related Work

ACDM stands on a large body of prior work. Stating that work plainly is part of the methodology, because the contribution is only credible once the borrowed parts are named.

5.1 Provenance modeling and W3C PROV

The W3C PROV family standardizes how to represent the entities, activities, and agents that produced a piece of data, along with constraints for validating provenance records. PROV is the reference vocabulary for describing lineage. ACDM treats PROV as a target representation: the provenance binding declared in an Entity Contract specifies what lineage an entity must be able to emit, and PROV is a natural format for emitting it. PROV describes what happened; it does not prescribe how to design a schema so that agents can consume it safely.

5.2 Provenance semirings and runtime annotation propagation

The provenance-semiring framework gives an algebra for propagating a single annotation, such as trust or cost, through relational operations at query time. It is the mathematical backbone for reasoning about how an annotation on inputs becomes an annotation on outputs. ACDM relies on this idea but operates earlier: where semirings compute propagation at runtime, ACDM asks the modeler to declare, at design time, how five

properties should combine, so that a semiring-style engine has rules to enforce. The algebra is the runtime mechanism; ACDM supplies the design-time intent.

5.3 Data contracts

The data-contracts movement formalizes agreements between data producers and consumers, usually covering schema, semantics, and service levels, and enforces them in pipelines. ACDM extends the spirit of data contracts to a new counterparty. A conventional data contract answers questions a downstream pipeline asks. An Entity Contract answers questions an autonomous agent asks, including one that traditional data contracts usually do not make central: what is this agent permitted to do with the data.

5.4 Catalogs, lineage, and observability

Data catalogs hold business definitions and metadata; lineage tools such as OpenLineage record how data sets relate; data-observability and the newer agent-observability tools, including OpenTelemetry GenAI conventions and tracing systems for language-model applications, capture what happened at runtime. These are complementary to ACDM and largely retrospective. ACDM is the design-time complement: a validator can read catalog tags and emit OpenLineage facets, so the methodology rides on the tools teams already run.

5.5 Modeling lineage: Kimball, Data Vault, data mesh

Dimensional modeling optimized schemas for analytical queries and human comprehension. Data Vault optimized for auditable, source-faithful warehousing with hubs, links, and satellites. Data mesh and the data-product movement shifted ownership to domains and treated data as a product with explicit quality guarantees. ACDM can be understood as an extension of this modeling lineage, aimed at a newer consumer: the autonomous agent. It reuses their instincts, especially the data-product framing, and adds the agent-specific governance header.

5.6 AI governance and human-in-the-loop control

The NIST AI Risk Management Framework and its Generative AI Profile give organizations a structured way to identify, measure, and manage AI risk. The OWASP Top 10 for LLM Applications names concrete failure classes that ACDM addresses directly, including sensitive information disclosure and excessive agency, and the companion OWASP Top 10 for Agentic Applications extends this to autonomous agent systems. Work on harms from increasingly agentic systems argues for controls on autonomy, oversight, and traceability. These frameworks are process and taxonomy guidance, and policy engines enforce who may reach which resource. They operate at the level of identity, system, or request; ACDM operates one layer earlier, on the intrinsic properties of the data, and connects those properties to an agent action ceiling.

5.7 The remaining gap

Across this prior work, governance metadata is represented, computed, or enforced, and several recent efforts increasingly recognize agent consumption as a distinct concern. What they do not yet provide is a unified design-time modeling methodology that combines entity-level contracts, multi-property composition rules, agent permission

ceilings, maturity assessment, and validator-based certification in one approach. Most existing techniques treat governance as something that travels with data at or after runtime, rather than as a discipline applied while an entity is modeled, before any data flows. That combination is the contribution ACDM proposes.

Prior approach	What it provides	Remaining gap for agent-safe modeling
W3C PROV	Standard vocabulary for representing lineage	No design-time rules for agent consumption
Provenance semirings	Runtime propagation of one annotation	Single-property, computed after design, not declared
Data contracts	Producer-consumer schema and SLA agreements	Limited or no explicit agent permission model
Catalogs / lineage / observability	Metadata and retrospective tracing	Describes the past, not design-time limits
Kimball / Data Vault / data mesh	Modeling for analysts and data products	Consumer is a human or pipeline, not an agent
NIST AI RMF, OWASP LLM / Agentic	Risk taxonomies and process guidance	Not attached to entities; do not resolve to permissions

Table 1. Prior work and the design-time gap ACDM addresses.

6. How ACDM Differs from Traditional Data Contracts

Because ACDM uses the word contract, it is worth stating plainly how it differs from the data-contracts practice, so the methodology is not read as a rename of an existing idea. Traditional data contracts focus on producer-consumer agreements: schema, semantics, data quality, availability, and service levels, usually enforced in pipelines for downstream teams. An Entity Contract keeps that spirit but serves a different consumer. It governs what an autonomous or semi-autonomous agent is allowed to do with the data, a question conventional data contracts usually do not make central.

Traditional data contracts ask whether data is well-formed and delivered as promised. ACDM asks what an AI agent is allowed to do with the data.

Traditional Data Contract	ACDM Entity Contract
Focuses on producer-consumer schema and service levels	Focuses on agent-safe consumption
Validates schema, quality, and availability	Validates trust, freshness, provenance, cost, risk, and action permission
Designed for pipelines and downstream teams	Designed for autonomous and semi-autonomous agents
Mostly checks whether data can be used	Checks what an agent is allowed to do with the data
Often enforced in data pipelines	Enforced through contracts, composition rules, permission ceilings, and validator certification

Table 2. Traditional data contracts compared with ACDM Entity Contracts.

7. The ACDM Methodology

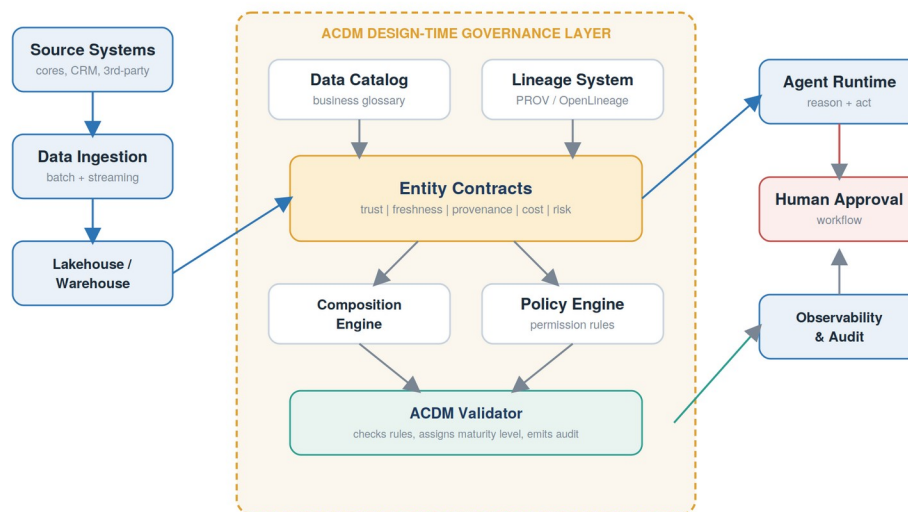
ACDM is a design-time methodology. It is applied while modeling entities, before agents consume them, and it produces artifacts that runtime systems can enforce. It is not a tracing tool and does not observe agents in production. The methodology has four parts that build on each other:

1. **The Entity Contract:** a machine-readable governance header attached to each important entity, declaring five agent-relevant properties plus ownership and exposure rules.
2. **Composition rules:** deterministic rules for how those properties combine when entities are joined, aggregated, enriched, or materialized.
3. **The agent-consumption permission model:** a mapping from an entity's resolved properties to a concrete permission level.
4. **The maturity model and validator:** a staged model that scores how far a data product has adopted ACDM, checked by a validator that emits an audit record.

The remainder of the paper defines each part, then shows how they fit a reference architecture, a reference implementation, and an adoption path. Figure 1 shows where the ACDM layer sits in a modern enterprise data platform, between the catalog and lineage tools and the agent runtime.

Figure 1. ACDM Conceptual Architecture

Where the Entity Contract layer sits inside a modern enterprise data platform



Traditional platforms stop at catalog + lineage. ACDM adds the contract, composition, permission, and certification layer in between.

Figure 1. ACDM conceptual architecture: the contract, composition, permission, and certification layer between catalog and agent.

Figure 2 shows the lifecycle each entity follows under the methodology, from drafting a contract through certification, publication, and scheduled review. Derived entities pass through the lifecycle again whenever a dependency's contract changes.

From draft through certification, publication, and scheduled review



An Entity Contract is a machine-readable governance header attached to an enterprise entity. It extends a normal entity definition, which already carries attributes, keys, and relationships, with the declarations an agent needs in order to consume the entity safely. The contract is authored at design time by the entity owner and is versioned alongside the schema.

1. `entity_name`: the canonical name of the entity.
2. `business_definition`: one sentence stating what the entity represents.
3. `primary_keys`: the keys that identify a record.
4. `trust_tier`: declared source reliability class.
5. `freshness_policy`: valid age, expiry behavior, and decay.
6. `provenance_binding`: required lineage depth and resolution.
7. `cost_class`: relative cost of materializing or querying.
8. `risk_class`: sensitivity, compliance exposure, and blast radius.
9. `allowed_agent_actions`: the permission ceiling for this entity.
10. `human_approval_required`: whether action requires review.
11. `downstream_exposure_rules`: how the entity may propagate to derived entities and outputs.
12. `owner`: the accountable team or person.
13. `review_frequency`: how often the contract is re-examined.

9

```
# Entity Contract: ACDM v1.0
entity_name: customer_profile
business_definition: >
  Authoritative identity and relationship attributes for a customer.
primary_keys: [customer_id]
trust_tier: T1 # T1 system-of-record .. T4 best-effort estimate
freshness_policy:
  max_age: 24h
  on_expiry: degrade # block | degrade | warn
  decay: none
provenance_binding:
  required_depth: source_of_record
  format: openlineage
cost_class: C2 # C1 cheap .. C4 expensive to materialize
risk_class: R3 # R1 public .. R4 highly sensitive / regulated
allowed_agent_actions: [read, summarize, recommend]
human_approval_required: true # for any action beyond recommend
downstream_exposure_rules:
  inherit_min_trust: true
  inherit_max_risk: true
  pii_masking: required
owner: customer-data-domain
review_frequency: quarterly
```

9. The Five Agent Properties

ACDM models five properties that determine how an agent may interact with an entity. Each follows the same practical structure: a definition, why agents need it, example values, what fails if it is missing, and how it affects permission.

9.1 Trust tier

Definition. A declared class for how reliable the source of an entity is, ranging from a system of record to a best-effort estimate. It is a design decision made by the owner, not a computed score.

Why agents need it. Agents cannot tell, from values alone, whether a number is authoritative. A clean decimal from a scraped estimate looks identical to one from the ledger. The trust tier tells the agent how much weight a value can bear.

Example values. T1 system of record; T2 governed derived; T3 third-party feed; T4 best-effort estimate.

Failure if missing. Without a trust tier, an agent treats an estimate as ground truth and acts on it with unwarranted confidence.

Permission effect. Higher trust raises the action ceiling. T1 can support autonomous action; T4 supports read only, all else equal.

9.2 Freshness policy

Definition. Declares the maximum valid age of the entity, what happens when that age is exceeded, and whether usefulness decays gradually or expires sharply. Resolved states are fresh, degraded, and expired.

Why agents need it. An agent acting on stale data is one of the most common and least visible failures. A declared freshness window lets the agent refuse or downgrade rather than guess.

Example values. max_age 24h, on_expiry block; max_age 7d, on_expiry degrade; no expiry for reference data.

Failure if missing. Without a freshness policy, a silently failed feed leads the agent to act on data that is days or weeks out of date.

Permission effect. Fresh permits the full ceiling; degraded caps at summarize; expired blocks action.

9.3 Provenance binding

Definition. Declares the lineage an entity must be able to produce, including how deep it must resolve and in what format. Resolved states are source-of-record, one-hop, unresolved, and missing.

Why agents need it. When an agent acts, someone will later ask why. Provenance binding guarantees the answer exists and resolves to a named source, not a dead end.

Example values. required_depth source_of_record; required_depth one_hop; format openlineage or PROV.

Failure if missing. Without provenance binding, a contested recommendation cannot be traced, and the organization cannot defend or correct it.

Permission effect. Source-of-record permits the full ceiling; one-hop permits recommend; unresolved caps at summarize; missing blocks action.

9.4 Cost class

Definition. Declares the relative expense of materializing, refreshing, or querying the entity, so agents and planners can budget their use of it.

Why agents need it. Agents can call data repeatedly inside a single reasoning loop. Without a cost signal, an agent may re-materialize an expensive aggregate dozens of times unnecessarily.

Example values. C1 cached and cheap; C2 routine query; C3 heavy join; C4 expensive external call.

Failure if missing. Without a cost class, agent loops drive unbounded compute spend and latency that no one anticipated at design time.

Permission effect. Cost does not make an action safe by itself. It constrains behavior only when a budget threshold is crossed, warning, throttling to read only, or blocking on budget.

9.5 Risk class

Definition. Declares sensitivity, compliance exposure, and blast radius: how much damage results if the entity is misused or exposed.

Why agents need it. Risk is the property that should most constrain autonomous action. An agent may freely summarize public reference data but must never act unattended on highly sensitive, regulated attributes.

Example values. R1 public; R2 internal; R3 confidential; R4 highly sensitive or regulated.

Failure if missing. Without a risk class, an agent may take an irreversible action on a regulated attribute that should always have required human review.

Permission effect. R1 can support autonomous action; R2 and R3 require human approval for consequential action; R4 caps at recommend with human approval and forces masked exposure.

10. ACDM Composition Rules

Entities are rarely consumed alone. They are joined, aggregated, enriched, and materialized into derived entities, and an agent usually reads the derivative. The central modeling error ACDM prevents is letting a derived entity inherit the best properties of its inputs when it should inherit the worst. Composition rules make property inheritance explicit and conservative by default: trust inherits the weakest contributing tier, freshness inherits the strictest or most expired dependency, provenance takes the union, cost accumulates, and risk inherits the highest class. Each rule is a design-time check, verifiable before any data flows. Rule statements name the risk class explicitly (R1 to R4) wherever risk is involved, so the rule numbers and the risk levels are never confused.

Rule	Statement	Example
R1 Trust floor	A derived entity's trust tier may not exceed the lowest trust tier among its inputs unless a named owner certifies an upgrade with evidence.	Joining a T1 profile with a T3 vendor score yields a T3 entity, not T1.
R2 Strictest freshness	A derived entity inherits the tightest expiry and the oldest effective age among its dependencies.	If one input expires in 24h and another in 7d, the derivative expires in 24h.
R3 Provenance union	A derived entity's provenance binding is the union of its inputs' bindings; it must resolve every contributing source.	An eligibility table must trace to profile, transactions, and the vendor score together.
R4 Cost accumulation	Cost class accumulates across materialization and query dependencies rather than resetting at each layer.	A C2 join over two C3 inputs is treated as at least C3, not C2.
R5 Risk ceiling	A derived entity inherits the maximum risk class among its inputs.	Any entity touching an R4 attribute is itself R4 until the attribute is removed or masked.
R6 Permission tightening	Agent permissions on a derivative may only be equal to or more restrictive than the strictest input permits.	If any input forbids autonomous action, the derivative forbids it too.

Rule	Statement	Example
R7 Expiry cascade	When an input expires, every derived entity that depends on it is treated as expired until refreshed.	A stale vendor feed expires the eligibility table that consumes it.
R8 Masking inheritance	Downstream exposure rules, including masking, propagate to every derivative and to agent-visible outputs.	If a profile requires PII masking, summaries built from it inherit masking.
R9 Certified upgrade	A property may be upgraded against the default only through a documented, owned certification recorded in the contract.	A reconciliation step may certify a T3 input as T2 with stated evidence.
R10 Conflict resolution	When composition yields a property below an agent's action threshold, the design must remedy it: split, refresh, mask, downgrade, or block.	If a join drops trust below the action floor, the model splits read-only fields from actionable ones.

Table 3. ACDM composition rules. Each rule is checkable at design time.

10.1 Worked composition example

Consider three inputs. `customer_profile` is T1, fresh, source-of-record, C1, R4. `third_party_risk_score` is T3, fresh, one-hop, C4, R2. `account_standing` is T1, fresh, source-of-record, C2, R3. Applying the composition rules, the derived `product_eligibility` entity resolves to T3 trust under Rule R1, 24h freshness under Rule R2, union provenance under Rule R3 (the vendor branch resolves only one-hop), C4 cost under Rule R4, and R4 risk under Rule R5. Trust at T3 caps the ceiling at recommend, and risk at R4 caps it at recommend with human approval. Therefore, the resolved ceiling is **recommend with human approval**. A naive join would have presented an apparently authoritative row and permitted autonomous action; ACDM downgrades it by construction.

11. Agent-Consumption Permission Model

The permission model is where ACDM goes furthest beyond what most prior art addresses. PROV and lineage describe the past. The permission model declares the future: given an entity's resolved properties, what an agent is allowed to do with it. Permissions form an ordered ladder, from most restrictive to least:

1. **Blocked:** the agent may not read the entity at all.
2. **Read only:** the agent may read values but not surface them to a user or act on them.
3. **Summarize only:** the agent may describe the entity in aggregate but not recommend or act.
4. **Recommend with disclaimer:** the agent may suggest an action, flagged as advisory.
5. **Recommend with human approval:** a suggestion that a person must accept before anything happens.
6. **Act with human approval:** the agent prepares an action that a person authorizes.
7. **Act autonomously:** the agent acts without review.

A permission level is the most permissive level allowed by every one of the five properties at once. The lowest property governs, consistent with the composition rules.

When two properties permit the same action type but require different controls, the stricter control applies. For example, if trust permits recommendation but risk requires human approval, the resolved ceiling becomes recommendation with human approval.

Figure 3 shows the decision flow, and Table 4 shows how each property pulls the ceiling down.

Figure 3. Agent-Consumption Permission Decision Flow

How the five properties resolve to a permission level



Figure 3. Agent-consumption permission decision flow.

Property	Strong	Adequate	Weak	Failing
Trust	Act autonomously	Act w/ approval	Recommend	Read only
Freshness	Act autonomously	Act w/ approval	Summarize	Blocked
Provenance	Act autonomously	Recommend	Summarize	Blocked
Cost	No restriction	No restriction	Budget warning / read only	Budget block / read only
Risk	Act autonomously	Act w/ approval	Recommend w/ approval	R4 unmasked: summarize

Table 4. Decision matrix. The agent's permission is the minimum ceiling across all five rows.

Read the matrix row by row for a given entity, then take the lowest cell. An entity with strong trust and provenance but only weak freshness lands at summarize, because freshness caps it there. This is intentional: the model is conservative so that a single weak property cannot be averaged away by strong ones. Cost is treated differently from the other four. It does not make an action safe by itself; it only constrains behavior when cost, latency, or resource usage crosses a defined threshold, which is why its strong and adequate cells impose no restriction. The risk row's failing state, **R4 unmasked**, denotes

highly regulated data that would be exposed without masking or approval controls; in that state the ceiling is capped at summarize only.

12. ACDM Maturity Model

The maturity model lets an organization measure adoption and lets an agent know how much it can trust the modeling itself. It has five levels. The Entity Contract lifecycle in Figure 2 maps onto these levels: an entity is annotated, composed, contracted, and finally certified as it climbs them.

Level	Description	Entry criteria	What an agent can safely do	Evidence
L0 Unmodeled	Entities carry no agent governance; agents consume blind.	No contracts exist.	Nothing safely; read-only at most.	A schema with no contract file.
L1 Annotated	Entities declare the five properties, but rules are not enforced.	Every important entity has a contract with all required fields.	Read and summarize, with caution.	Contract files passing a field completeness check.
L2 Composed	Composition rules are applied across dependencies.	Derived entities inherit properties per Table 3.	Recommend, where composed properties allow.	A composition report showing inherited properties.
L3 Contracted	Agent permissions are modeled and enforced; conflicts are resolved.	Every agent-facing entity has a permission level and a conflict remedy.	Act with human approval, where permitted.	An enforced permission matrix and resolved conflicts.
L4 Certified	A validator certifies the product and emits an audit record.	Validator passes; audit metadata emitted to catalog and lineage.	Act autonomously, only for entities that earn it.	A signed validator report and emitted facets.

Table 5. ACDM maturity model, with entry criteria, safe agent behavior, and example evidence per level.

12.1 Self-assessment checklist

1. Does every agent-facing entity have a contract with all required fields? (Level 1)
2. Do derived entities inherit properties by the composition rules, checked in CI? (Level 2)
3. Does every agent-facing entity carry an enforced permission level and a conflict remedy? (Level 3)
4. Does a validator certify the product and emit an audit record on each release? (Level 4)

13. Reference Architecture

ACDM is designed to fit a modern enterprise data platform without replacing any of it. The components below already exist in most organizations; ACDM adds the contract, composition, permission, and validator layer between the catalog and the agent runtime, as shown in Figure 1.

1. **Source systems** feed raw data, each with an inherent trust and risk profile.
2. **Data ingestion** lands data in batch and streaming modes.
3. **Lakehouse or warehouse** stores modeled entities.
4. **Data catalog** holds business definitions and hosts contract metadata.
5. **Entity contracts** declare the five properties and permissions at design time.
6. **Lineage system** resolves provenance, emitting PROV or OpenLineage.
7. **Composition engine** applies the inheritance rules to derived entities.
8. **Policy engine** enforces permission levels at consumption time.
9. **Agent runtime** consumes entities within the declared limits.
10. **Human approval workflow** handles actions that require review.
11. **Observability and audit** records what the agent did and why, closing the loop.

Figure 4 contrasts a traditional pipeline, where governance is inferred at runtime, with an ACDM pipeline, where it is declared at design time and enforced.

Figure 4. Traditional Modeling vs. ACDM
Same pipeline, different consumer assumptions

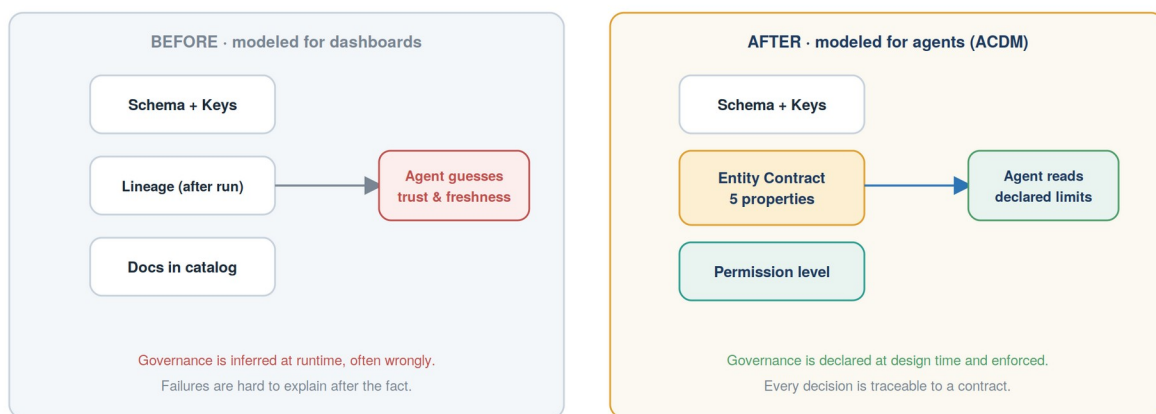


Figure 4. Traditional modeling versus ACDM for the same pipeline.

14. Reference Implementation

ACDM is enforceable by a validator that reads Entity Contracts, checks them against the rules, assigns a maturity level, and emits an audit report. The validator can read contracts from YAML sidecar files, dbt metadata, Unity Catalog or Microsoft Purview tags, or catalog annotations, and can emit metadata to OpenLineage, OpenTelemetry, or an enterprise catalog. **This pseudocode is illustrative and not production-ready.** A production validator

would require stronger typing, dependency-graph traversal, policy integration, and catalog-specific adapters. Tier and class labels are converted to numeric ranks before comparison, and the field names match the Entity Contract in Section 8.

```
# Lower trust rank = more trustworthy; higher risk rank = more sensitive.
TRUST_RANK = {"T1": 1, "T2": 2, "T3": 3, "T4": 4}
RISK_RANK = {"R1": 1, "R2": 2, "R3": 3, "R4": 4}

REQUIRED = ["entity_name", "business_definition", "primary_keys",
            "trust_tier", "freshness_policy", "provenance_binding",
            "cost_class", "risk_class", "allowed_agent_actions",
            "human_approval_required", "downstream_exposure_rules",
            "owner", "review_frequency"]

def validate(entity, deps):
    findings = []
    # Level 1: completeness
    missing = [f for f in REQUIRED if f not in entity]
    if missing:
        findings.append(("L1", "missing required fields", missing))

    # Level 2: conservative composition (derived entities only)
    if deps:
        own = TRUST_RANK[entity["trust_tier"]]
        worst = max(TRUST_RANK[d["trust_tier"]] for d in deps)
        if own < worst and not entity.get("certified_upgrade"):
            findings.append(("R1", "trust better than weakest input"))
        if RISK_RANK[entity["risk_class"]] < max(RISK_RANK[d["risk_class"]] for d in
        deps):
            findings.append(("R5", "risk lower than an input"))
        own_age = to_seconds(entity["freshness_policy"]["max_age"])
        tightest = min(to_seconds(d["freshness_policy"]["max_age"]) for d in deps)
        if own_age > tightest:
            findings.append(("R2", "freshness window looser than an input"))

    # Level 3: permission ceiling consistency
    ceiling = permission_ceiling(entity) # derived from the decision matrix
    declared = max(entity["allowed_agent_actions"], key=rank_action)
    if rank_action(declared) > rank_action(ceiling):
        findings.append(("R6", "declared action exceeds the property ceiling"))

    level = assign_maturity(entity, deps, findings)
    emit_audit(entity, level, findings) # catalog / OpenLineage facet
    return AuditReport(entity["entity_name"], level, findings)
```

The validator runs in continuous integration so that a contract regression fails the build, the same way a failing test does. Its audit report is the artifact a governance team reviews and an auditor can later inspect.

15. Worked Example: Synthetic Validation Study

All entities, properties, and numbers in this section are synthetic and illustrative. They do not describe any real organization, employer, client, or confidential system.

This section applies ACDM to a synthetic regulated financial-services assistant that recommends next-best actions to service staff. The scope is four core entities and three derived entities. The input contracts are shown in Table 6.

Entity	Trust	Freshness	Provenance	Cost	Risk
customer_profile	T1	24h fresh	source-of-record	C1	R4
transaction_summary	T2	24h fresh	source-of-record	C3	R3
third_party_risk_score	T3	168h fresh	one-hop	C4	R2
account_standing	T1	24h fresh	source-of-record	C2	R3

Table 6. Input contracts for the four core entities (illustrative).

Three derived entities are built from these inputs. `product_eligibility` joins `customer_profile`, `third_party_risk_score`, and `account_standing`. `next_best_action_features` joins `product_eligibility` with `transaction_summary` and `account_standing`. `recommended_offer` is the proposed action derived from `next_best_action_features`. Every derived entity inherits R4 from the customer record and T3 from the vendor branch, so none is eligible for autonomous action; each resolves to recommend with human approval, as shown in Table 7.

Derived entity	Trust	Freshness	Cost	Risk	Resolved ceiling
product_eligibility	T3	24h	C4	R4	Recommend w/ approval
next_best_action_features	T3	24h	C4	R4	Recommend w/ approval
recommended_offer	T3	24h	C4	R4	Recommend w/ approval

Table 7. Composed properties and resolved ceilings for the derived entities (illustrative).

Modeling these contracts prevents four classes of unsafe action by construction, summarized in Table 8.

Unsafe action	How ACDM prevents it
An autonomous credit-limit increase on a high-risk record	The R4 and T3 resolution caps the ceiling at recommend with human approval (R1, R5).
Acting on a stale vendor score after the feed silently fails	Expiry cascades to every downstream entity, dropping them to summarize or blocked (R7).
Surfacing unmasked personal data to a customer-facing channel	Masking propagates from <code>customer_profile</code> to all derivatives and outputs (R8).
A planning loop re-materializing an expensive vendor aggregate	The C4 cost class triggers a budget warning and a read-only cap once the budget is crossed.

Table 8. Unsafe actions prevented in the synthetic study.

Table 9 shows the synthetic before-and-after outcome of adopting ACDM for this scope. The figures show the shape of the evidence, not a measured result; an adopting team replaces them with counts it can defend.

Measure	Before	After
Entities classified with full contracts	0 of 7	7 of 7
Missing freshness policies found	3	0
Provenance gaps found	2	0
High-risk (R4) entities restricted from autonomous action	0	4
Derived entities downgraded due to weaker trust	0	3
Maturity level reached	Level 0	Level 3

Table 9. Synthetic before-and-after outcomes for the example scope. Illustrative only.

16. Practical Adoption Guide

Organizations should adopt ACDM on one data product first, prove the value, then widen. The recommended path:

1. Select one agent-facing data product where an agent already reads or acts on data.
2. Identify the core entities and their derivatives.
3. Define an Entity Contract for each, declaring the five properties.
4. Apply the composition rules to every derived entity.
5. Assign agent permission levels using the decision matrix.
6. Validate the maturity level and resolve the conflicts the validator flags.
7. Add validator checks to continuous integration so a contract regression fails the build.
8. Integrate the validator with the catalog and lineage tools.
9. Add a human approval workflow for actions above the recommend ceiling.
10. Move from Level 1 to Level 4 gradually, widening to more products as the pattern proves out.

16.1 A first 30/60/90 day plan

Window	Focus	Target outcome
Days 1-30	Pick one product. Inventory its core and derived entities. Draft Entity Contracts and classify the five properties with owners.	Level 1 (Annotated): every entity has a complete contract.
Days 31-60	Encode composition rules and run them in CI. Assign permission ceilings and resolve flagged conflicts.	Level 2 to 3: composition enforced, permissions modeled.
Days 61-90	Wire the policy engine and human-approval workflow. Emit audit facets to the catalog and lineage. Review with governance.	Level 3 to 4 on the pilot, with an audit trail and a repeatable pattern.

Table 10. An illustrative first 30/60/90 day adoption plan for a single data product.

17. Relationship to Prior Art and Limitations

ACDM does not invent provenance, trust propagation, lineage, metadata tagging, or data contracts. Its contribution is the combination of a design-time modeling discipline, a five-property Entity Contract, multi-property composition rules, an agent-consumption permission model, a maturity model, and a validator-based certification approach. Table 11 states what is borrowed and what is added.

Prior work	What it already does	What ACDM borrows	What ACDM adds
W3C PROV	Represents lineage	Provenance format	Design-time binding requirement
Provenance semirings	Propagates one annotation at runtime	Propagation intuition	Declared multi-property composition
Data contracts	Producer-consumer agreements	Contract framing	Agent permission model
Catalogs / lineage	Hold metadata, trace data	Hosting and resolution	Five-property header and rules
Kimball / Vault / mesh	Model for analysts and products	Data-product instinct	Agent as first-class consumer

Table 11. Honest separation of borrowed and added contributions.

17.1 Limitations

- Trust tiers are assigned by owners and are therefore partly subjective; the methodology standardizes the act of declaring them, not the judgment behind them.
- Contracts that are not maintained decay into inaccurate metadata, which can be worse than none.
- ACDM does not cryptographically verify provenance; it requires that provenance be resolvable, which is a weaker guarantee than attestation.
- ACDM does not replace runtime monitoring. It reduces the chance of unsafe action by design but does not observe what agents actually do.
- ACDM needs tooling support to be enforced at scale; without a validator and policy engine it is advisory.
- Thresholds, such as what freshness or risk level permits which action, vary by domain and regulation and must be set locally.
- Autonomous action remains inappropriate for some high-risk contexts regardless of how well the data is modeled.
- The methodology needs more real-world pilots; the evidence in this paper is a synthetic worked example.

18. Future Work

1. An OpenLineage facet proposal that carries the five ACDM properties as a standard, interoperable facet.

2. A reference implementation for Unity Catalog, Microsoft Purview, and dbt metadata.
3. Policy-engine integration so the permission ceiling is enforced at consumption time on common stacks.
4. Cryptographic provenance attestation, so contracts can require attested rather than merely resolvable lineage.
5. A benchmark for agent-safe data products, scoring how many unsafe actions a given contract set prevents.
6. Cross-domain empirical validation, testing whether the conservative defaults hold outside financial services.
7. Alignment with the NIST AI Risk Management Framework and enterprise AI governance programs, so ACDM fits existing compliance processes rather than competing with them.

19. Conclusion

Many enterprises are now proving that agents can query their data. The harder question is whether agents can act on it safely. That shift demands a modeling discipline aimed at the agent as a consumer, not just at the analyst or the dashboard. ACDM proposes that discipline. By requiring each important entity to declare its trust, freshness, provenance, cost, and risk, by composing those properties conservatively, and by mapping them to an explicit permission level that a validator can certify, ACDM helps a data team move from *“an agent can query our data”* to *“an agent can safely consume and act on governed data.”* Agentic AI requires data engineering to move from verifying the structure of data to bounding what may be done with it. The methodology borrows heavily and claims narrowly. Its value is not a new theory but a practical, defensible way to design data that autonomous systems can be trusted to use.

References

- [1] World Wide Web Consortium (W3C). PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.
- [2] World Wide Web Consortium (W3C). Constraints of the PROV Data Model (PROV-CONSTRAINTS). W3C Recommendation, 30 April 2013.
- [3] Green, T. J., Karvounarakis, G., and Tannen, V. Provenance Semirings. Proc. 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS), 2007, pp. 31-40. doi:10.1145/1265530.1265535.
- [4] Inmon, W. H. Building the Data Warehouse, 4th Edition. Wiley, 2005.
- [5] Kimball, R., and Ross, M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd Edition. Wiley, 2013.
- [6] Linstedt, D., and Olschimke, M. Building a Scalable Data Warehouse with Data Vault 2.0. Morgan Kaufmann, 2015.
- [7] Dehghani, Z. Data Mesh: Delivering Data-Driven Value at Scale. O'Reilly Media, 2022.
- [8] Jones, A. Driving Data Quality with Data Contracts. Packt Publishing, 2023.
- [9] OpenLineage. OpenLineage Specification. LF AI & Data Foundation. <https://openlineage.io>.

- [10] OpenTelemetry. Semantic Conventions for Generative AI. Cloud Native Computing Foundation. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>.
- [11] National Institute of Standards and Technology (NIST). Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, January 2023. doi:10.6028/NIST.AI.100-1.
- [12] National Institute of Standards and Technology (NIST). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1, July 2024. doi:10.6028/NIST.AI.600-1.
- [13] OWASP. OWASP Top 10 for Large Language Model Applications (2025). OWASP GenAI Security Project, 2025. <https://genai.owasp.org>.
- [14] OWASP. OWASP Top 10 for Agentic Applications (2026). OWASP GenAI Security Project, Agentic Security Initiative, December 2025. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>.
- [15] Chan, A., Salganik, R., Markelius, A., et al. Harms from Increasingly Agentic Algorithmic Systems. Proc. 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT '23), 2023, pp. 651-666. doi:10.1145/3593013.3594033.

Appendix A. Publication-Readiness Checklist

1. Name and acronym reviewed for clarity, with the full name used on first mention and the clinical-data-management overlap noted.
2. All references confirmed; OWASP Agentic Applications verified against the canonical resource page.
3. Figures rendered, numbered sequentially, and never referenced before they are introduced.
4. Tables numbered sequentially and captioned below each table.
5. Worked-example numbers clearly marked synthetic and illustrative throughout.
6. No confidential or employer-specific data included anywhere.
7. Novelty stated as a unified combination rather than an absolute first.
8. Validator pseudocode labeled illustrative and checked against the composition rules and the decision matrix.
9. Entity names and contract fields formatted consistently in monospace.
10. Soft-hyphen and encoding artifacts removed so the text copies cleanly.
11. External reviewer feedback obtained before formal publication.

Appendix B. Entity Contract Full Template

```
# Agent-Data Entity Contract Schema v1.0
metadata:
  entity_name: <string>
  business_definition: <one sentence>
  owner: <domain or team>
  review_frequency: <quarterly | monthly | on_change>
schema:
  primary_keys: [<column>, ...]
```

```

pii_masking: <required | not_required>
agent_properties:
  trust_tier: <T1 | T2 | T3 | T4>
  freshness_policy:
    max_age: <duration, e.g. 24h, 7d>
    on_expiry: <block | degrade | warn>
    decay: <none | linear | threshold>
  provenance_binding:
    required_depth: <source_of_record | one_hop>
    format: <openlineage | w3c_prov>
  cost_class: <C1 | C2 | C3 | C4>
  risk_class: <R1 | R2 | R3 | R4>
governance_rules:
  allowed_agent_actions: [read, summarize, recommend, act]
  human_approval_required: <true | false>
  downstream_exposure_rules:
    inherit_min_trust: <true | false>
    inherit_max_risk: <true | false>
certified_upgrades:
  - property: <trust | risk | ...>
    from: <value>
    to: <value>
    evidence: <link or reference>
    approved_by: <owner>

```

Appendix C. Validator and Maturity Self-Assessment Worksheet

Use the following checks to place a data product on the maturity scale and to drive the validator's findings.

Check	Pass criterion	Level
Field completeness	Every agent-facing entity has all thirteen required contract fields.	1
Composition validation	Derived entities inherit trust, freshness, provenance, cost, and risk per the CR rules, checked in CI.	2
Permission consistency	Each entity's declared actions do not exceed its resolved ceiling; conflicts have a remedy.	3
Certification and audit	A validator pass emits a signed audit record and facets to the catalog and lineage on each release.	4

Table 12. Validator checks mapped to maturity levels.